

Lab exercises

Part 08 —  Retrieval-Augmented Generation · 08-rag/lab/EXERCISES.md

Lab — Retrieval-Augmented Generation

1. Break a fact on purpose, then unbreak it (1.5h)

Using `../code/chunking_strategies.py` as your base, write a new multi-paragraph document (your own topic, 5-6 paragraphs, at least 400 words) with one fact planted so it straddles an unlucky fixed-size chunk boundary — the same trick the file already does with the Library of Alexandria's scroll count.

- **1a.** Show the fact gets split by `fixed_size_chunk` at some chunk size you choose deliberately.
- **1b.** Show `recursive_chunk` at the same nominal size keeps it intact.
- **1c.** Show `sentence_window_chunk` keeps the fact's anchor sentence intact by construction, and print both the anchor and the window handed to the LLM.

Deliverable: your document, plus the three chunk listings, plus two sentences explaining *why* recursive and sentence-window both solved it via a different mechanism (one packs whole sentences; one never chunks sub-sentence at all).

Checks: `fact_is_intact()`-style boolean must be `False` for your fixed-size run and `True` for the other two. If you can't make the fixed-size case fail, your document's paragraph structure is too convenient — pick a longer sentence with the critical fact past the midpoint.

2. Chunk-size sweep on your own labeled task (2h)

Adapt `run_chunk_size_sweep()` to a small labeled retrieval task of your own: at least 5 short documents, each built as filler + one answer sentence + filler (same construction as the file's `ARTICLES`), and one query per document targeting its answer sentence exclusively.

- **2a.** Sweep at least 4 chunk sizes (including "whole document").
- **2b.** Report top-1 accuracy AND average margin at each size — accuracy alone saturates on a clean toy task; margin is what shows the trade-off before it becomes an outright miss.
- **2c.** Identify the chunk size where margin first drops below 0.15, and explain in one paragraph what that would mean for a real corpus of hundreds of similar documents (not this toy one).

Deliverable: your sweep table plus the paragraph from 2c. **Checks:** margin must fall monotonically (or very close to it) as chunk size grows — if it doesn't, your filler isn't diluting the way the module's version does; check that your filler shares vocabulary across all documents.

3. Prove hybrid beats either system alone — on queries you designed to break it (2h)

Using `../code/hybrid_search_and_rerank.py` as a base, add 4 new documents and 6 new queries to the corpus: 3 queries that should clearly favor BM25 (exact codes/IDs/names) and 3 that should clearly favor dense (full paraphrase, zero shared vocabulary with the target document).

- **3a.** Run BM25, Dense, Hybrid (RRF), and +Rerank on your new queries. Report MRR per system, split by query type, the same way the file's own `main()` does.
- **3b.** Find (or construct) one query where Hybrid RRF is NOT within 0.1 MRR of the best individual system for that query. Explain why, using the fusion formula — what would have to be true about the two systems' individual rankings for RRF to fail this way?
- **3c.** Modify the RRF `k` constant from 60 to 5 and rerun. Explain what changed and why (hint: re-read what `k` controls in the docstring, then check whether the change matches your prediction).

Deliverable: your extended corpus/queries, the MRR tables, and written answers to 3b and 3c. **Checks:** on your BM25-favoring queries, BM25's MRR should exceed Dense's by a wide margin, and vice versa for your dense-favoring queries — if not, your queries aren't actually testing what you think they are.

4. Verify the eval metrics yourself, independent of the code (1.5h)

Do **not** run `../code/rag_eval_metrics.py` for this exercise yet. On paper (or in a fresh scratch script), given:

```
relevant = {"A": 2, "B": 1, "C": 0, "D": 1}    # grades: 0=not relevant, 1=somewhat, 2=highly
ranked   = ["C", "A", "E", "D", "B"]          # a retrieval system's top-5, in order
```

- **4a.** Compute Recall@3 by hand. Show your work.
- **4b.** Compute Reciprocal Rank by hand (remember: 1-indexed ranks).
- **4c.** Compute NDCG@3 by hand: DCG@3 using $(2^{\text{rel}} - 1) / \log_2(\text{rank}+1)$, then IDCG@3 from the ideal ordering of the *known* relevance grades, then divide.
- **4d.** Now run `recall_at_k`, `reciprocal_rank`, and `ndcg_at_k` from the actual file on this exact input and confirm your hand-computed numbers match to 3 decimal places.

Deliverable: your worked math for 4a-4c, plus the confirmation output from 4d. **Checks:** if your hand calculation and the code disagree, the bug is almost always a 0-index vs 1-index mistake in the rank, or forgetting the $2^{\text{rel}} - 1$ (not plain `rel`) in the DCG formula. Find it before moving on — don't just trust the code.

5. Design a faithfulness-breaking answer, then explain why the checker misses it (2h)

Using `check_faithfulness` [08-rag/code/rag_eval_metrics.py] and its `CONTEXT_CHUNKS`, write three new candidate answers to `QUESTION`:

- **5a.** A **fully faithful but heavily paraphrased** answer — correct according to the context, but using almost none of the context's actual vocabulary (e.g., "you're covered" instead of "eligible for a refund"). Run it

through `check_faithfulness` and report the score.

- **5b.** A **negated, unfaithful** answer — take a true sentence from the context and flip its meaning with a single word ("is" → "is not", or invert a number), keeping every other word identical. Run it and report the score.
- **5c.** For both 5a and 5b, explain in a short paragraph *why* the lexical-overlap checker gets the wrong verdict, referencing the specific caveat in the docstring that predicts each failure. Then describe, in concrete terms (not "use an LLM"), what signal an NLI-based judge would use that this checker structurally cannot compute.

Deliverable: your two answers, their scores, and the two explanation paragraphs. **Checks:** 5a should score noticeably LOWER than its true faithfulness deserves (a false negative); 5b should score HIGH despite being false (a false positive). If neither happens, your examples aren't adversarial enough — increase the paraphrase distance in 5a, or make the negation more surgical in 5b.

6. Capstone — a tiny end-to-end RAG pipeline with a real eval (4h)

Build a minimal pipeline over a small corpus of your choosing (10-20 short documents — your own notes, a set of Wikipedia summaries, your company's public docs, anything with real variety):

- **6a.** Chunk it with the structure-aware or recursive chunker from Exercise 1's file.
- **6b.** Index it with BM25 (from `hybrid_search_and_rerank.py`) — a real dense embedding model is a bonus, not required (TF-IDF/LSA from the file is an acceptable stand-in and keeps this fully offline).
- **6c.** Write 8-10 queries against your corpus with graded relevance judgments (0/1/2) — judge them yourself, honestly, before you look at what any system retrieves.
- **6d.** Run BM25 alone and Hybrid+Rerank (if you have a second retrieval signal) or two different chunk sizes for BM25-only, and score both with Recall@3, MRR, and NDCG@3 using `rag_eval_metrics.py`'s functions.
- **6e.** Write 3 generated answers (you can write these by hand, playing the role of the LLM, or actually call one) to 3 of your queries, and score their faithfulness against the retrieved context.

Deliverable: a short script or notebook producing (1) your retrieval metrics table comparing at least two configurations, (2) your faithfulness scores for the 3 answers, and (3) a one-paragraph recommendation: which configuration you'd ship, and the single biggest weakness you'd fix next if you had one more day.

Check: your two configurations must actually differ in at least one metric — if they're identical, you haven't changed enough between them to learn anything. The recommendation paragraph must cite specific numbers from your own table, not "system B seemed better."